

Age-of-Information Minimization for Data Aggregation in Energy-Harvesting IoTs

Bingkun Yao
City University of Hong Kong
bingkun.yao@cityu.edu.hk

Mun Choon Chan
National University of Singapore
chanmc@comp.nus.edu.sg

Hong Gao*
Zhejiang Normal University
honggao@zjnu.edu.cn

Zhe Jiang*
Southeast University
zhejiang.uk@gmail.com

Nan Guan*
City University of Hong Kong
nanguan@cityu.edu.hk

Abstract—Energy Harvesting (EH) technology has emerged to prolong the lifetime of Internet of Things (IoT) devices. However, in EH-IoTs, the reliance on external energy sources introduces challenges in maintaining up-to-date information. To quantify data freshness in such systems, researchers have introduced the Age-of-Information (AoI) metric, which measures the time elapsed since the generation of the most up-to-date information received by the user. Consequently, the problem of AoI minimization has been studied extensively in EH-IoTs to ensure timely data delivery. While data aggregation is a fundamental task for IoTs, existing works on AoI minimization in EH-IoTs have only considered scenarios where sensory data is updated by individual source nodes. The problem has not been investigated for data aggregation, in which the sensory data is aggregated from multiple source nodes. In this paper, we study the problem of AoI minimization for Data Aggregation in EH-IoTs. To address this problem, we propose an energy-adaptive node scheduling algorithm consisting of both offline scheduling and online adjustment. Extensive simulations and testbed experiments verify the high performance of our algorithm in terms of AoI minimization and energy efficiency.

I. INTRODUCTION

In recent years, Energy-Harvesting (EH) technology has emerged as a crucial solution for Internet of Things (IoT) systems [1]–[3]. By enabling nodes to harvest energy from the ambient environment, EH technology significantly prolongs the lifetime of IoT networks and provides a promising approach for green computing [4].

In EH-IoTs, nodes rely on harvested energy to operate, which introduces unique challenges in maintaining the freshness of sensory data. Due to the intermittent nature of energy harvesting, nodes can only sense and transmit data when sufficient energy is accumulated, leading to outdated information. To tackle this issue, researchers have introduced Age of Information (AoI) [5] to quantify data freshness. To be specific, AoI is defined as the time difference between the present and the timestamp of the most up-to-date sensory data received by the user.

To ensure data freshness in EH-IoTs, the problem of AoI minimization in EH-IoTs is of great importance. While extensive research has focused on this problem [9]–[30], these studies concentrate on the scenario where each interested data item is updated by individual nodes (referred to as *individual-source data collection*).

Data aggregation, as one of the fundamental tasks in IoTs, presents a distinct scenario where each interested data item is the aggregation of data from multiple nodes. For instance, in marine environmental monitoring, obtaining the average temperature of a sea area requires aggregating temperature readings from multiple sensor nodes distributed across the region.

The AoI metric in the scenario of data aggregation is quite different from that in individual-source data collection. In individual-source data collection, the data captured by a single node can be directly utilized by the user. Thus, the AoI metric for individual-source data

collection *only reflects the freshness of data collected by a single node*. In contrast, for the scenario of data aggregation, the data of interest is aggregated from multiple nodes' samples. A data item is considered to be received only when all its constituent data have been transmitted, and its timestamp is determined by the timestamps of its aggregated data. Therefore, the AoI in this scenario *represents the freshness of data jointly captured by multiple nodes*, making existing solutions designed for individual-source collection inapplicable to data aggregation contexts.

Moreover, the characteristic of energy harvesting makes AoI minimization in data aggregation scenarios particularly challenging in EH-IoTs. This is because different nodes typically have distinct energy harvesting rates, resulting in diverse duty cycles. When multiple data items need to be updated by nodes with different energy harvesting rates, coordinating these nodes to effectively reduce the overall AoI becomes a complex problem.

In this paper, we investigate the problem of **AoI** minimization for Data Aggregation in EH-IoTs (**AoI-Agg** for short), which has not been studied in the literature. The main contributions are as follows: First, we formalize the AoI-Agg problem that aims to minimize the AoI metric in the context of data aggregation. Second, we propose a bipartite approach: an offline algorithm to generate the basic schedule, and an online adjustment scheme to maintain AoI performance under fluctuating energy harvesting rates. Finally, we validate our proposed method through extensive simulations and testbed experiments, demonstrating its effectiveness in terms of both AoI and energy efficiency.

II. PROBLEM STATEMENT

A. System Model

Network Model. The network consists of multiple sensor nodes $N = \{n_1, n_2, \dots\}$ that monitor a set of targets $T = \{t_1, t_2, \dots\}$. Here, a target refers to any kind of physical entity or phenomenon that users might be interested in. For example, the temperature of a water body. Each node n covers a subset of targets, denoted as $\Gamma(n)$, while $Cover_node(t)$ represents the set of nodes that can monitor target t .

The system operates in discrete time, with the entire working divided into K slots with equal length. In each slot t_x ($t_x \in [1, K]$), a node n operates in either *active* or *idle* state. When active, n transmits the sensed data of all the targets in $\Gamma(n)$, and the data is sampled by n at the t_s -th slot. We represent this action as a *sub-instance* denoted by a triple $s = (n, t_x, t_s)$, where t_x and t_s are called the *sending-time* and *sub-timestamp* of s .

We introduce the concept of sub-instance because the sensed data of a target is aggregated from the data of multiple nodes, and a single sub-instance typically represents only a partial observation of the target state. We denote the set of all the sub-instances generated by n as $S(n)$, which represents the working schedule of node n . Each

* Corresponding authors.

node can transmit at most one sub-instance per timeslot, meaning that for any two sub-instances $s_1, s_2 \in S(n)$, $s_1.t_x \neq s_2.t_x$.

Energy model. For each node $n \in N$, we define its energy parameters as follows:

- $B_n(i)$ represents n 's available energy at the start of slot i .
- $EH_n(i)$ represents the energy harvesting rate during slot i .
- $B_{max}(n)$ specifies the maximum energy storage capacity.
- $E_c(n)$ indicates the energy consumption for one active slot.

$E_c(n)$ depends on $|\Gamma(n)|$ and the size of the sensed data of each target in $\Gamma(n)$. We assume $E_c(n) \leq B_{max}(n)$ to ensure the node can complete at least one active slot when fully charged. During each slot i , if node n is active, it first consume $E_c(n)$ units of energy, then harvests $EH_n(i)$ units from the environment.

Data Aggregation Model. For each target $t \in T$, each data instance of t needs to be aggregated from the data sampled by $Agg_num(t)$ different nodes. In real-world applications, usually redundant nodes are deployed to improve the coverage of the network. Thus, $Agg_num(t) \leq |Cover_node(t)|$.

We define a **combination** Com as a set of sub-instances that can form a complete data instance of a target t . A valid combination must satisfy these conditions:

A1: Causality Constraint. For each sub-instance $s \in Com$, we have $s.t_x \geq s.t_s$; This ensures that transmission occurs after sampling.

A2: Node Diversity Constraint. Com consists of $Agg_num(t)$ sub-instances, each from a distinct node in $Cover_node(t)$.

A3: Temporal Consistency Constraint. For any $s, s' \in Com$, $|s.t_s - s'.t_s| \leq \delta$, where δ denotes the maximum allowable temporal deviation between sub-instances. Sub-instances with excessive temporal differences cannot be aggregated as they may not represent a consistent state.

A combination Com has two key attributes:

- **Arrival time** $Ar(Com) = \max\{s.t_x | s \in Com\}$. This denote the time when Base Station (BS) receives the data instance aggregated from Com , at which all sub-instances in Com complete their transmission.
- **Timestamp** $Ts(Com) = \min\{s.t_s | s \in Com\}$. The timestamp of the aggregated data instance is the smallest sub-timestamp of all its sub-instances.

Let $Q(t)$ denote the set of all valid combinations for target t . We restrict each sub-instance to be aggregated at most once for each target, namely, for any $Com, Com' \in Q(t)$, $Com \cap Com' = \emptyset$;

B. Definition of AoI-Agg Problem

For each target t , at slot i , the AoI of t is the difference between the current time and the timestamp of the most up-to-date t 's data instance maintained at the BS:

$$AoI(t, i) = i - \max\{Ts(Com) | Ar(Com) \leq i \ \& \ Com \in Q(t)\}$$

Following the convention in existing works on AoI minimization [14], [21], [36], at the start of slot 1, BS maintains an initial data instance with timestamp 0 for each target t .

The objective of AoI-Agg is to compute a working schedule for all the nodes such that the average AoI of all the targets throughout the entire working duration is minimized. We formally define the problem as follows:

Input: 1) Network parameters $N, T, \Gamma(n)$ for each $n \in N, K$;

2) Energy parameters $B_{max}(n), E_c(n), B_n(0)$ for each $n \in N$, the predicted value of $EH_n(i)$ for each $n \in N$ and $i \in [1, K]$;

3) $Agg_num(t)$ for each $t \in T$ and δ .

Output: Working schedule $S(n)$ for each $n \in N$, Combination set $Q(t)$ for each $t \in T$ such that the following is minimized:

TABLE I
SYMBOLS AND NOTATIONS

Notation	Description
N	The set of all energy-harvesting (EH) nodes
T	The set of all interested targets
$\Gamma(n)$	The set of targets that can be covered by node n
$Cover_node(t)$	The set of nodes that cover target t
K	The length of the whole working duration
$s = (n, t_x, t_s)$	A sub-instance of node n with sending-time t_x and sub-timestamp t_s
$S(n)$	The set of all sub-instances of n
$B_n(i)$	The energy level of n at the start of the i -th slot
$EH_n(i)$	The EH rate of n during the i -th slot
$E_c(n)$	The energy consumption for an active slot of n
$Agg_num(t)$	The aggregation number of target t
Com	A combination corresponding to a data instance
$Ar(Com)$	The arrival time of the data instance aggregated from Com , $Ar(Com) = \max\{s.t_x s \in Com\}$
$Ts(Com)$	The timestamp of the data instance aggregated from Com , $Ts(Com) = \min\{s.t_s s \in Com\}$

$$AVG_AoI = \frac{\sum_{i=1}^K \sum_{t \in T} AoI(t, i)}{K|T|}$$

Assuming perfect knowledge of $EH_n(i)$ is impractical. Therefore, we use predicted values of $EH_n(i)$ as problem input. In our evaluation, we employ Pro-Energy [35] to estimate these values. During schedule execution, a node n remains idle if its available energy falls below $E_c(n)$, even when scheduled to be active.

The AoI-Agg problem is NP-Hard, as a special case of it is equivalent to the 2-DSC problem [43], which has been proven to be NP-complete. Therefore, AoI-Agg is NP-hard. We omit the detailed proof due to space limitations.

III. AM-AGG: COMPUTATION OF A BASIC SCHEDULE

In this section, we propose the **AoI Minimization for Data Aggregation (AM-Agg)** algorithm to compute a basic working schedule for each node. Overall, AM-Agg iterates on the timeline starting from the first slot till the last slot, dividing the entire working duration into several periods, and computing the node working schedule within each period. The i -th iteration consists two phases: (1) Determine the scale of the i -th period so that at least one new data instance could be updated for each target within the i -th period; (2) Scheduling nodes for the i -th period to enhance the energy usage and reduce AoI.

A. Phase I: Determine the Scale of the i -th Period

Let $[start_i, end_i]$ denote the i -th period from slot $start_i$ to end_i . As AM-Agg algorithm process periods in chronological order, $start_1 = 1$ and $start_i = end_{i-1} + 1$. Within each period, we need to find a combination Com_t for each target $t \in T$ to ensure all target data are updated. To minimize the data staleness, we aim to minimize end_i while satisfying this update requirement. The sub-problem of this phase can be defined as:

Input: $start_i$.

Output: end_i , a new combination Com_t for each $t \in T$ such that $Ar(Com_t) \in [start_i, end_i]$ and end_i is minimized.

Now consider this sub-problem. We define a **full-coverage** as a node set denoted by FC such that for each target t , $|Cover_node(t) \cap FC| \geq Agg_num(t)$. Also, let $Ready(FC, j)$ denote the first slot after or equal to j when all nodes in FC are expected to enough energy to become active¹. We use FC_{min} to denote the full-

¹This time is predicted based on the input energy parameters. e.g., If $B_n(10) = 20$, $E_c(n) = 100$ and the predicted $EH_n(k) = 10$ for $k \geq 10$, n is expected to have enough energy to become active at slot 18.

coverage that minimizes $Ready(FC, start_i)$. Given FC_{min} , a solution for the above subproblem can be constructed as follows:

1) For each node $n \in FC_{min}$, we have a new sub-instance $s_{new}(n)$ where $s_{new}(n).t_x = s_{new}(n).t_s = Ready(FC_{min}, start_i)$. This is because n is expected to have enough energy to be active at slot $Ready(FC_{min}, start_i)$.

2) For each target $t \in T$, Com_t is composed of new sub-instances from nodes that are can cover t :

$$Com_t = \{s_{new}(n) | n \in Cover_node(t) \cap FC_{min}\}$$

and $|Com_t| = Agg_num(t)$.

Through above method, end_i is minimized, and $end_i = Ready(FC_{min}, start_i)$. Hence, to solve the subproblem above, we only need to compute FC_{min} . Also, to reduce energy consumption, FC_{min} should not contain redundant nodes, namely, any proper subset of FC_{min} is not a full-coverage. FC_{min} is computed as follows:

1) Initiate FC_{min} as N . Then for each $n \in FC_{min}$, if n does not have enough energy to become active in residual working duration, remove n from FC_{min} .

2) Sort FC_{min} by the descending order of $Node_ready(n, start_i)$, which is the first slot j after or equal to $start_i$ when $B_n(j)$ is expected to reach $E_c(n)$.

3) For each node $n \in FC_{min}$, if $FC_{min}/\{n\}$ is a full-coverage, delete n from FC_{min} .

If FC_{min} is not a full-coverage, there is no feasible solution for the above sub-problem, and we simply set $end_i = K$. This indicates that the AM-Agg will end after the i -th iteration.

B. Phase II: Scheduling within the i -th Period

Phase I only consider nodes in FC_{min} , and these nodes are only scheduled to be active at slot end_i . To further improve the energy usage and reduce the AoI, Phase II of AM-Agg tries to find more new combinations within the i -th period, including two steps: (1) Find the set of targets T_{new} to be updated. (2) Compute a new combination for each target in T_{new} such that the arrival time and the timestamp of each combination is minimized and maximized respectively. Two steps are repeated until new combinations could not be found.

Before the detailed description of this phase, to allocate new active slots for each node n , AM-Agg maintains the interval $[Fc(n), Lc(n)]$ during which n could become active, which satisfies the following:

1) $Fc(n), Lc(n) \in [La(n)+1, end_i-1]$, and $La(n)$ denotes the last slot before end_i ² at which n has been scheduled to be active. This is because the schedule is computed in chronological order. If n has not been scheduled to be active, $La(n) = 0$.

2) $B_n(Fc(n))$ and $B_n(Lc(n))$ are expected to exceed $E_c(n)$, and If $n \in FC_{min}$, then n becoming active in that slot is not expected to cause $B_n(end_i)$ to fall below $E_c(n)$. This is to ensure the scheduling complies with energy constraints.

If no slots satisfy above conditions, $Fc(n) = Lc(n) = -1$, which indicates that we cannot assign new active slots for n in the i -th period. Next, we describe two steps of Phase II in detail.

Step 1: Compute T_{new} . We need to ensure that a new data instance could be updated for each target in T_{new} . Firstly, let N_{new} denote the set of nodes that could transmit new sub-instance (i.e., $Fc(n) \neq -1$). Then, T_{new} is the set of all the targets that N_{new} can cover, namely,

$$T_{new} = \{t | |Cover_node(t) \cap N_{new}| \geq Agg_num(t)\}$$

If $T_{new} = \emptyset$, no new combinations could be found during the i -th period and Phase II ends.

Step 2: Compute a new Com_t for each $t \in T_{new}$. This step has two substeps: (1) To update the sensory data more quickly, $Ar(Com_t)$

²As the data instance of each target is updated to BS at slot end_i , Phase II does not consider the schedule for slot end_i .

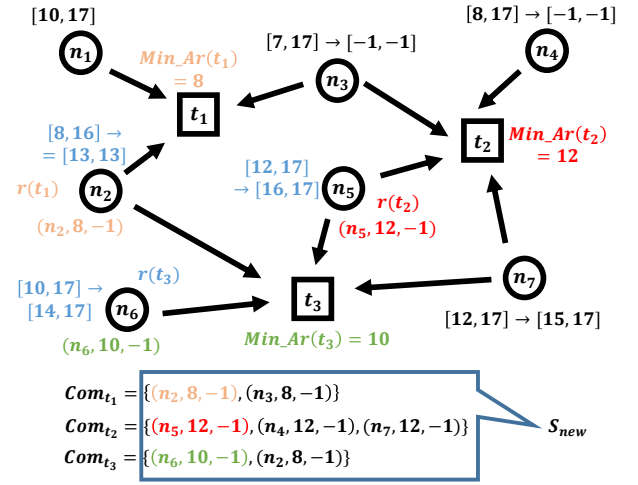


Fig. 1. An example of minimizing $Ar(Com_t)$ for each $t \in T_{new}$, and $T_{new} = \{t_1, t_2, t_3\}$.

is minimized for each $t \in T_{new}$. (2) Given $Ar(Com_t)$, $Ts(Com_t)$ is maximized to ensure that the sensory data is received by the BS as soon as being collected.

Substep 1: Minimize $Ar(Com_t)$. Com_t consists of newly-computed sub-instances, whose set is denoted by S_{new} . The subproblem of this substep is:

Input: T_{new}, N_{new} .

Output: $S_{new}, Com_t \subseteq S_{new}$ for each $t \in T_{new}$ such that $Ar(Com_t)$ is minimized.

We illustrate how to solve the above problem by the example shown in Fig. 1, where $T_{new} = \{t_1, t_2, t_3\}$, $N_{new} = \{n_1, \dots, n_7\}$. $Agg_num(t_1) = Agg_num(t_3) = 2$ and $Agg_num(t_2) = 3$. The $[Fc(n), Lc(n)]$ for each node is also marked in the figure.

Firstly, according to the definition of arrival time in Sec. II-A, for each $t \in T_{new}$, $Ar(Com_t)$ is lower bounded by $Min_Ar(t) = Fc(r(t))$, in which $r(t)$ is the node covering t with the $Agg_num(t)$ -th largest $Fc(n)$. To minimize $Ar(Com_t)$, AM-Agg adds a sub-instance $(r(t), Min_Ar(t), -1)$ ³ into Com_t and S_{new} . As highlighted in red in Fig. 1, $Agg_num(t_2) = 3$ and among 4 nodes covering t_2 , n_5 has the third largest Fc value (i.e., 12). Thus, $Min_Ar(t_2) = 12$, $r(t_2) = n_5$ and the sub-instance $(n_5, 12, -1)$ is added into Com_{t_2} and S_{new} . Com_{t_1} and Com_{t_3} are also processed similarly (Highlighted in orange and green). Besides, as highlighted in blue, after n_5 , n_2 and n_6 are scheduled to be active at slot 12, 8 and 10, their $[Fc(n), Lc(n)]$ are updated.

Then, AM-Agg computes the remaining sub-instances for each Com_t . The basic idea is that for each node $n (n \neq r(t))$ covering t , the sending-time of n 's sub-instance should be as close as possible but no more than $Min_Ar(t)$. This is to make $Ts(Com_t)$ closer to $Ar(Com_t)$ while ensuring that $Ar(Com_t) = Min_Ar(t)$. The example in Fig. 1 details this point as described below. Mention that targets in T_{new} are considered in the ascending order of $Min_Ar(t)$ (For our example, it is $t_1 \rightarrow t_3 \rightarrow t_2$):

(1) Firstly, consider Com_{t_1} . Since $Fc(n_1) = 10 > Min_Ar(t_1) = 8$ and there is no sub-instances of n_1 in S_{new} , we cannot assign n_1 's sub-instance to Com_{t_1} . Also, since $Fc(n_3) = 7 < 8$, n_3 could become active at slot 8. Thus, AM-Agg adds a new sub-instance $(n_3, 8, -1)$ into Com_{t_1} . After this, $Fc(n_3) = Lc(n_3) = -1$.

(2) Secondly, as for Com_{t_3} , since $Fc(n_5), Fc(n_7) > Min_Ar(t_3) = 10$, and the sending-times of their sub-instances in S_{new} are larger

³-1 means the sub-timestamp of this sub-instance has not been determined.

Algorithm 1: Substep 1: Minimize $Ar(Com_t)$

Input: N_{new} , T_{new} , $Fc(n)$ and $Lc(n)$ for each $n \in N_{new}$;
Output: A new combination Com_t for each $t \in T_{new}$, a set of new sub-instances S_{new} ;

- 1: $S_{new} \leftarrow \emptyset$;
- 2: **for** each $t \in T_{new}$ **do**
- 3: $r(t) \leftarrow$ the $Agg_num(t)$ -th ranked node in $Cover_node(t) \cap N_{new}$ in the ascending order of $Fc(n)$;
- 4: $Min_Ar(t) \leftarrow Fc(r(t))$;
- 5: $Com_t \leftarrow \{(r(t), Fc(r(t))), -1\}$, $S_{new} \leftarrow S_{new} \cup Com_t$;
- 6: Sort T_{new} in the ascending order of $Fc(r(t))$;
- 7: **for** each $t \in T_{new}$ **do**
- 8: $Scan(t) \leftarrow \emptyset$; /*The candidate sub-instances of Com_t */
- 9: **for** each $n \in Cover_node(t) \setminus r(t)$ **do**
- 10: Compute a candidate sub-instance s of n to maximize $s.t_x$ satisfying $s.t_x \leq Min_Ar(t)$, $Scan(t) \leftarrow Scan(t) \cup \{s\}$;
- 11: Select $Agg_num(t)-1$ sub-instances with the largest sending-time from $Scan(t)$, and add them to Com_t ;
- 12: $S_{new} \leftarrow S_{new} \cup Com_t$;

than $Min_Ar(t_3)$ (i.e., that of n_5 is 12), we must consider n_2 . Then, because $Fc(n_2) = 13 > 10$, to ensure $Ar(Com_{t_3}) = Min_Ar(t_3)$, AM-Agg adds the existing sub-instance $(n_2, 8, -1)$ in S_{new} into Com_{t_3} .

(3) Finally, consider Com_{t_2} . Since $Fc(n_4), Fc(n_7) < Min_Ar(t_2) = 12$, AM-Agg adds $(n_4, 12, -1)$ and $(n_7, 12, -1)$ into Com_{t_2} . As for n_3 , since $Fc(n_3) = Lc(n_3) = -1$, we could not find a new sub-instance of it. Moreover, although the sending-time of n_3 's sub-instance in S_{new} is less than $Min_Ar(t_2)$ ($8 < 12$), it is not as close to $Min_Ar(t_2)$ as those of n_4 and n_7 . Thus, n_3 's sub-instance is not added into Com_{t_2} .

Algorithm 1 summarizes the procedure of Substep 1. Through this method, $Ar(Com_t)$ is minimized for each $t \in T_{new}$.

Substep 2: Maximizing $Ts(Com_t)$ by computing the sub-timestamps of sub-instances in Com_t . For a target $t^* \in T_{new}$, before the computation of $Ts(Com_{t^*})$, the sub-timestamp of some Com_{t^*} 's sub-instances may have been determined. This is because Com_{t^*} may intersect with other Com_t ($t \in T_{new}$). Thus, we only need to consider those sub-instances with undetermined sub-timestamps, whose set is denoted by $UD(Com_{t^*})$. The subproblem is defined as:

Input: S_{new} , $UD(Com_{t^*}) = \{s | s \in Com_{t^*} \& s.t_s = -1\}$;
Output: $s.t_s$ for $s \in UD(Com_{t^*})$ such that $Ts(Com_{t^*})$ is maximized.

We solve this by maximizing the sub-timestamp of sub-instances in $UD(Com_{t^*})$ in the ascending order of their sending-times.

For each sub-instance $s \in UD(Com_{t^*})$, we aim to maximize its sub-timestamp. To ensure Com_{t^*} is valid, Com_{t^*} must satisfy conditions A1, A2, and A3. Algorithm 1 ensures that $|Com_{t^*}| = Agg_num(t^*)$, thus satisfying A2. Our task now is to maximize the sub-timestamp of s while meeting A1 and A3.

1) Firstly, ensure A3 (Temporal Consistency Constraint) is satisfied. Since $Com_{t^*} \subseteq S_{new}$, all sub-instances in S_{new} may limit the value of $s.t_s$. An example is shown in Fig. 2, in which $\delta = 1$, $t^*, t_1, t_2 \in T_{new}$ and $s_{1-5} \in S_{new}$. $s_2.t_s$ has been determined as 7. Consider $s_3.t_s$. As highlighted in blue in Fig. 2, since $s_2.t_s = 7$ and $s_2, s_4 \in Com_{t_2}$, $s_4.t_s \leq 7 + \delta = 8$. Similarly, $s_5.t_s \leq 8 + \delta = 9$. Finally, $(s_3.t_s)$ cannot exceed $9 + \delta = 10$. Besides, as highlighted in red, although $s_1.t_s$ is undetermined, it is at most 6 since $s_1.t_x = 6$. Similar to the limit imposed by s_2 , s_1 restricts $s_3.t_s$ from exceeding 9 as highlighted in red in Fig. 2.

We can formalize the above restrictions as an undirected graph $G = (V, E)$, in which each $s \in S_{new}$ corresponds to a vertex $v(s)$ and if any two sub-instances s_1, s_2 belongs to a combination, $(v(s_1), v(s_2)) \in E$. For any $s' \in S_{new}$, if $v(s)$ and $v(s')$ are connected through a k -hop path, the constraints of s' on $s.t_s$ have the following two cases:

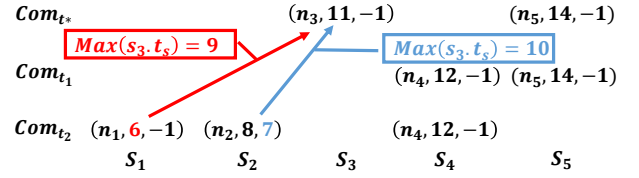


Fig. 2. The restrictions of s_1 and s_2 on $s_3.t_s$.

(a) $s'.t_s$ is determined before $s.t_s$. As highlighted in blue in Fig. 2, $s.t_s \in [s'.t_s - k\delta, s'.t_s + k\delta]$.

(b) Otherwise, we have two subcases: (1) $s'.t_x < s.t_x$. Since $s'.t_s \leq s'.t_x$, as highlighted in red in Fig. 2, $s.t_s \leq s'.t_x + k\delta$; (2) Otherwise, since $s.t_s \leq s.t_x$ and $s.t_x \leq s'.t_x$, we have $s.t_s \leq s'.t_x + k\delta$, indicating that s' does not limit $s.t_s$.

2) Secondly, to satisfy A1 (Causality Constraint), $s.t_s \leq s.t_x$.

To summarize above (1) and (2), when computing $s.t_s$ for a $s \in UD(Com_{t^*})$, for each sub-instance $s' \in S_{new}$, we have $s.t_s \in [R_{min}(s'), R_{max}(s')]$:

$$R_{min}(s') = \begin{cases} \text{Max}\{1, s'.t_s - h(s', s)\delta\}, & \text{If } s'.t_s \neq -1 \\ 1 & \text{Otherwise} \end{cases}$$

$$R_{max}(s') = \begin{cases} \text{Min}\{s'.t_s + h(s', s)\delta, s.t_x\} & \text{If } s'.t_s \neq -1 \\ \text{Min}\{s'.t_x + h(s', s)\delta, s.t_x\} & \text{If } s'.t_s == -1 \& s'.t_x < s.t_x \\ s.t_x & \text{Otherwise} \end{cases}$$

in which $h(s, s')$ is the minimum hop count from $v(s)$ to $v(s')$ in G . Finally, $s.t_s$ is the largest value that satisfies the restrictions of all other sub-instances in S_{new} :

$$s.t_s = \text{Max}\left\{i | i \in \bigcap_{s' \in (S_{new} \setminus \{s\})} [R_{min}(s'), R_{max}(s')]\right\}$$

The method above solves the subproblem of this substep optimally, and $Ts(Com_{t^*})$ is maximized.

IV. REAL-TIME SCHEDULE ADJUSTMENT

AM-Agg computes a basic working schedule based on the predicted EH rates. When the actual EH rate drops below the estimated, some nodes may not have enough energy to become active in some predetermined active slots. To tackle this issue, we propose an online schedule adjustment scheme. The basic idea is that in case of energy insufficiency, a node will only become active during slots that could reduce more AoI. This scheme contains two steps:

- (1) For each node n , divide node n 's sub-instances into some priorities, with higher priority ones able to reduce more AoI.
- (2) During the working duration, n will adjust its schedule according to its real-time energy status, in which higher-priority sub-instances are transmitted first.

Prioritize All the Sub-instances. Assuming there are p different priorities levels. Starting from the highest priority (Level p), the scheme selects the sub-instances for each priority level in order from highest to lowest priority (i.e., From p to 1), in which level i is selected as follows:

- (1) Initialize an empty set D .
- (2) Each time select a combination not in S from the combinations of all the targets, such that if this combination does not exist, the AVG_AoI would increase the most. Then, Add this combination to D , and set all sub-instances in this combination as i .
- (3) Repeat (2) until level i account for $1/p$ of all sub-instances of all the nodes.

Through the method above, sub-instances that are more critical to AoI minimization have higher priority.

Real-time Schedule Adjustment. Nodes adjust their working schedule based on real-time energy status and priority level of sub-instances. Specifically, at the start of slot i , if node n has been scheduled to become active by AM-Agg, then:

(1) n estimates its available energy for the remaining working duration, which is denoted by $AE_n(i)$ as follows:

$$AE_n(i) = B_n(i) + PE_n(i, K)$$

in which $PE_n(i, K)$ stands for the real-time estimated value of n 's total harvestable energy from slot i to K ⁴.

(2) n adjusts its schedule. The basic idea is to transmit more high-priority sub-instances while satisfying the energy constraints. Specially, n will transmit sub-instances with priority higher than b , which is minimized such that n 's energy consumption within the remaining working duration does not exceed $AE_n(i)$:

$$b = \underset{j \in [1, p]}{\text{ArgMin}} \left\{ AE_n(i) \geq \left| \bigcup_{k \in [j, p]} \text{Lev}(i, k) \right| \times E_c(n) \right\}$$

in which $\text{Lev}(i, k)$ denotes the set of sub-instances with priority level k and sending-time in $[i, K]$. If the priority level of n 's sub-instance at slot i is no less than b , n will transmit this sub-instance. Otherwise, just stay idle at slot i .

Through the schedule adjustment scheme, when nodes' energy level is lower than expected, those high-priority combinations that can reduce more AoI will still be updated, thus ensuring the AoI performance. This also works when nodes have different EH rates, as high-priority sub-instances are transmitted by nodes with both high and low energy. This scheme is evaluated in Sec. V-B.

V. EVALUATION

A. Simulation

This section evaluates the proposed algorithm through simulations. During each simulation, firstly, we use Pro-Energy [35] to predict the EH rate of each node. Then, the schedule of each node is computed by AM-Agg based on the predicted EH rate. During the working duration, each node adjusts its schedule using the scheme of Sec. IV.

Simulation Parameters. In the network, there are $|N| = 100$ nodes deployed to cover $|T| = 80$ targets. The average value of $|Cover_node(t)|$ and $Agg_num(t)$ is 7.5 and 2.5 respectively. The length of the working duration is $K = 1200$ and $\delta = 4$.

As for energy parameters, the simulation applies the energy trace from an indoor solar energy dataset [40] for $EH_n(i)$. The length of each slot is one second. Same as [41], to model heterogeneous EH rates among nodes, a random noise $\pm 50\%$ is added to the $EH_n(i)$. $E_c(n)$ is computed according to CC2500 datasheet [42], and each node needs 30 slots on average to harvest the energy required for one active slot.

Evaluation Metrics. Apart from the AoI metric (i.e., AVG_AoI), to explore the energy efficiency, we discuss the number of all combinations and *useful* combinations, in which a combination Com_t is considered to be *useful* if the AoI of t is reduced when Com_t is updated to BS.

Baselines. We compare our protocol with the lower bound of AVG_AoI , which is computed by the following equation:

$$Low_AVG_AoI = \sum_{i \in T} \frac{a_1[(a_1 - 1)p(t) + a_1 + a_2 + 1]}{2K|T|},$$

⁴The energy estimation method here is the same as that for predicting $EH_n(i)$ as the input of AM-Agg (i.e., Pro-Energy [35]). Note that as i increases, the time scale to be predicted decreases, thus improving the prediction accuracy. e.g., In our simulation, when $i = 0.1K$ and $0.9K$, the average prediction errors of $PE_n(i, K)$ are 18.8% and 11.5% respectively.

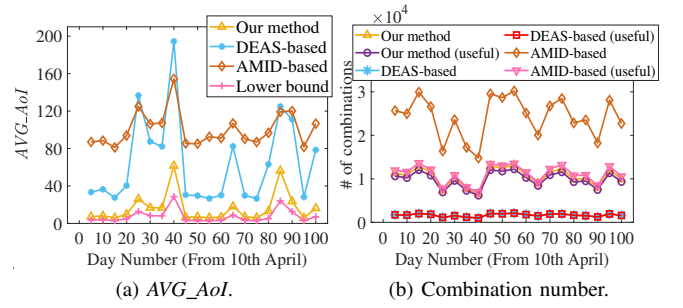
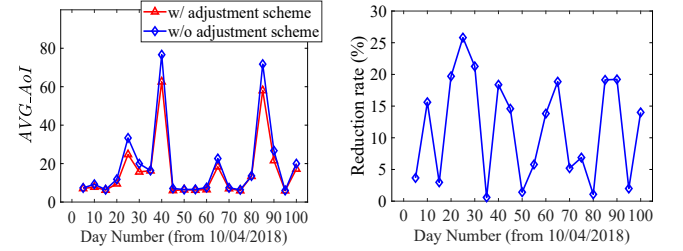


Fig. 3. Average performance of all working durations.



(a) The performance with and without adjustment scheme. (b) The reduction rate of average AoI after using the adjustment scheme.

Fig. 4. The impact of schedule adjustment scheme.

in which $a_1 = \lfloor \frac{K}{p(t)+1} \rfloor$, $a_2 = K \bmod (p(t)+1)$ and $p(t) = \sum_{n \in Cover_node(t)} \frac{B_n(0) + \sum_{i=1}^K EH_n(i)}{E_c(n) \times Agg_num(t)}$.

So far, AoI-Agg has not been studied. The most similar recent works are AIMD [22] and DEAS [31], which studied the problem of AoI minimization for individual-source data collection (see Sec. I) and minimum-latency data aggregation, respectively. Hence, we use them as baselines for comparison. Note that since the data of each target is updated only once by DEAS, it is iterated starting from the first slot to the end of the working duration in our evaluation. Also, following assumptions in [22] and [31], when evaluating AMID and DEAS, we assume perfect knowledge of $EH_n(i)$.

Simulation Results. This part shows the result of the simulation conducted on the energy data starting from 10/04/2018 to 19/07/2018, in which each data point represents the average of five consecutive days (i.e., 180 working durations). As shown by Fig. 3(a), our protocol outperforms two baselines, in which its performance is the most close to the lower bound (About $1.9 \times$ lower bound on average), and the average AoI is reduced by 75% and 84% compared to the DEAS and AMID respectively.

Comparison with DEAS: In Phase I and Substep 1 of Phase II, AM-Agg algorithm computes the schedule with the objective of minimizing the data reception time. This improves the frequency of data updates. This is shown in Fig. 3(b): our method has more combinations than DEAS. In comparison, DEAS does not consider the energy level of nodes, resulting in long data update intervals.

Comparison with AMID: In Substep 1 of Phase II, for a combination Com_t , the sending-time of each sub-instance in Com_t is made as close as possible to $Ar(Com_t)$. Also, through Substep 2 of Phase II, the timestamp of Com_t is also maximized. All of above not only facilitate fast data updates but also maximizes the freshness of the updated data, making more than 97% of combinations useful (Shown in Fig. 3(b)). In contrast, AMID could only ensure rapid data transmission without considering the data aggregation, leading to uncertainty in the freshness of its updated data.

In summary, different parts of AM-Agg play distinct roles: the calculation of FC_{min} of Phase I and the minimization of $Ar(Com_t)$ in Substep 1 of Phase II ensure the harvested energy is *fully* utilized,

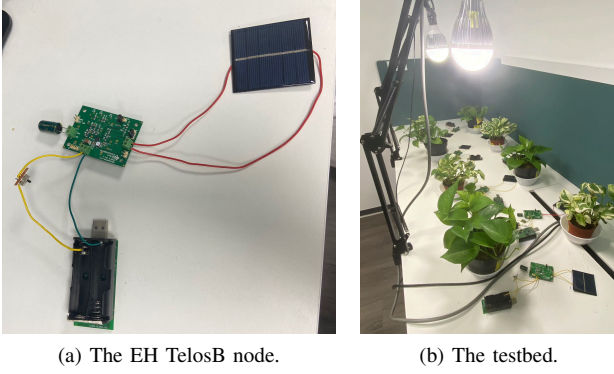


Fig. 5. Experimental hardware.

while the maximization of $Ts(Com_t)$ in Substep 2 of Phase II ensures the energy is *wisely* utilized. These improve energy efficiency, enabling our algorithm to achieve high performance in terms of AoI.

B. The Impact of Schedule Adjustment Scheme

We evaluate the impact of schedule adjustment scheme by comparing the performance with and without applying it. As shown in Fig. 4, the scheme reduces the average AoI by 11.5% on average compared to using the AM-Agg alone. As we observe, the difference in the proportion of combinations that could not be updated due to energy constraints, with and without the scheme, is only 1.52% on average. Thus, the main reason for the effectiveness is that the scheme prioritizes combinations that could reduce more AoI when energy is insufficient. Also, as shown in Fig. 4(b), the scheme is more effective when AoI is large. This is because there is relatively less harvestable energy under this condition, leading to more combinations not being updated, thus increasing the optimization space for the scheme.

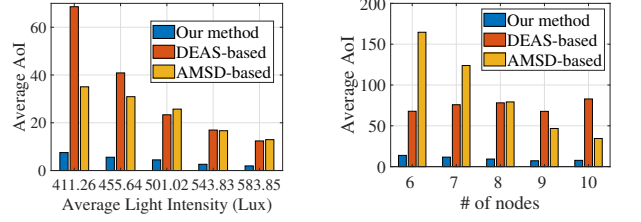
C. Testbed evaluation

To verify the simulation results, we also conduct experimental studies on real-world EH testbeds.

Experimental Settings. First, we uniformly deploy 10 EH TelosB nodes on the 120cm×70cm testbed (Fig. 5(a)), in which each node is equipped with a 6cm×5cm solar panel and TI BQ25504-EVM674 EH evaluation module [38]. We use two LEDs whose light intensity could be adjusted among 10 discrete levels as the energy source. As shown in Fig. 5(b), an indoor agriculture scenario is simulated, in which some plants are placed on the testbed such that some nodes are in shadow while other nodes are exposed directly to the light.

The whole working duration contains 1000 timeslots and the length of one slot is one second. During each execution, the light intensity is adjusted every 100 slots. We assume there are 8 targets to be monitored, and the average value of $|\Gamma(n)|$ and $Agg_num(t)$ is 6 and 2.5. The size of each sub-instance is 10 bytes, which contains the temperature data (4 bytes), the humidity data (4 bytes) and the sub-timestamp (2 bytes). During each active slot, the node will transmit its sub-instance two times with a transmission power of -11dBm. The default average light intensity at each node is set to be 411.26 Lux, in which it takes 25 idle slots on average for each node to harvest the energy needed for an active slot.

Experimental Results. We evaluate the performance on AoI in terms of the ambient light condition and nodes number. As illustrated by Fig. 6(a), as the ambient light intensity increases, nodes can harvest more energy, which increases the update frequency of all the targets. Thus, the performance of all protocols increases. As shown by Fig. 6(b), the performance of our method and AMID increases with the number of nodes. This is because as the nodes number grows, a target could be covered by more nodes and the sensed data is updated more



(a) AVG_AoI vs. ambient light condition. (b) AVG_AoI vs. nodes number.

Fig. 6. Results of testbed experiments.

frequently. As for DEAS, since the average value of $|\Gamma(n)|$ remains unchanged, the number of nodes required to cover all the targets shows no obvious change as nodes number increases. Under this condition, as DEAS only schedules the nodes in a full-coverage to become active, nodes number has little influence on its performance.

It can be seen that our method reduces the average AoI by 86% and 89% on average compared with DEAS and AMID respectively. This aligns well with our simulation results.

VI. DISCUSSION

For the scenario when targets have different weights. In Substep 1 of Phase II of AM-Agg, $Ar(Com_t)$ s for all targets in T_{new} are minimized simultaneously. This is also similar for Substep 2. Thus, AM-Agg tends to reduce the AoI of multiple targets at the same time, rather than minimizing the AoI of some targets at the expense of increasing the AoI of others. Hence, our method is still effective for scenarios where target weights are different. To validate this, we evaluate our method based on the setup of Sec. V-A, and the weight of targets varies between 0.1 and 3 (with an average of 1). The results show that the weighted AVG_AoI of our method is only $1.75\times$ lower bound on average. This is similar to the results in Sec. V-A.

About transmission conflict. In real-world EH-IoTs, nodes generally require a relatively long time for EH before they can operate. For example, according to [40], on average, a solar-powered node needs about 120 seconds to harvest enough energy to send 200 bytes of data, which could be transmitted in just 8 milliseconds. Thus, the slot length in EH-IoTs is much longer compared with conventional communication networks, usually ranging from several seconds to minutes [44]–[48]. For an EH-IoT consisting of hundreds of nodes, within each slot, we can even allocate a TDMA sub-slot to each node, thereby avoiding the transmission conflicts. Therefore, our study does not need to take transmission conflicts into account.

VII. CONCLUSION

In this paper, we study the AoI minimization problem for the scenario of data aggregation in EH-IoTs for the first time. We propose both an energy-adaptive scheduling algorithm and a real-time schedule adjustment scheme to solve the problem. Simulations and testbed experiments verify the high performance of our method in terms of AoI and energy efficiency.

ACKNOWLEDGEMENT

We would like to thank the anonymous reviewers for their helpful feedback. This work is supported by the National Natural Science Foundation of China (Grants No. 61632010, U22A2025), the Hong Kong General Research Fund (Grant No. 11208522), the National Natural Science Foundation of China (Grant No. 62472086), the Basic Research Program of Jiangsu (Grants No. BK20243042, BG2024010), and the Start-up Research Fund of Southeast University (Grant No. RF1028624005). This research receives support from China Scholarship Council.

REFERENCES

- [1] K.S. Adu-Manu, N. Adam, C. Tapparello, et al., "Energy-harvesting wireless sensor networks (EH-WSNs): A review," in *ACM Transactions on Sensor Networks*, vol. 14(2): 1-50, 2018.
- [2] G. Zhang, W. Zhang, et al., "Energy-Delay Tradeoff for Dynamic Offloading in Mobile-Edge Computing System With Energy Harvesting Devices," in *IEEE Transactions on Industrial Informatics*, vol. 14, no. 10, pp. 4642-4655, 2018.
- [3] Z. Cai, Q. Chen, T. Shi, T. Zhu, K. Chen and Y. Li, "Battery-Free Wireless Sensor Networks: A Comprehensive Survey," in *IEEE Internet of Things Journal*, vol. 10, no. 6, pp. 5543-5570, 2023.
- [4] S. Lambert, W. Van Heddeghem, W. Vereecken, B. Lannoo, D. Colle, and M. Pickavet, "Worldwide electricity consumption of communication networks," in *Opt. Exp.*, vol. 20, no. 26, pp. B513-B524, 2012.
- [5] S. Kaul, R. Yates and M. Gruteser, "Real-time status: How often should one update?," in *INFOCOM* 2012, pp. 2731-2735.
- [6] A.M. Bedewy, Y. Sun, S. Kompella and N.B. Shroff, "Age-optimal sampling and transmission scheduling in multi-source systems," in *Mobihoc* 2019, pp. 121-130.
- [7] R. Talak, S. Karaman, and E. H. Modiano, "Optimizing information freshness in wireless networks under general interference constraints," in *IEEE/ACM Trans. Netw.*, vol. 28, no. 1, pp. 15-28, 2020.
- [8] C. Li, S. Li, Y. Chen, Y. Thomas Hou and W. Lou, "AoI Scheduling with Maximum Thresholds," in *INFOCOM* 2020, pp. 436-445.
- [9] B. T. Bacinoglu, Y. Sun, et al., "Achieving the Age-Energy Tradeoff with a Finite-Battery Energy Harvesting Source," in *ISIT* 2018, pp. 876-880.
- [10] A. Arafat, J. Yang, et al., "Age-Minimal Transmission for Energy Harvesting Sensors With Finite Batteries: Online Policies," in *IEEE Transactions on Information Theory*, vol. 66, no. 1, pp. 534-556, 2020.
- [11] O. Ozel, "Timely Status Updating Through Intermittent Sensing and Transmission," in *ISIT* 2020, pp. 1788-1793.
- [12] S. Feng and J. Yang, "Age of Information Minimization for an Energy Harvesting Source With Updating Erasures: Without and With Feedback," in *IEEE Transactions on Communications*, vol. 69, no. 8, pp. 5091-5105, 2021.
- [13] Y. Liu, Z. Chang, G. Min and S. Mao, "Average Age of Information in Wireless Powered Mobile Edge Computing System," in *IEEE Wireless Communications Letters*, vol. 11, no. 8, pp. 1585-1589, 2022.
- [14] F. Chiarriotti et al., "Query Age of Information: Freshness in Pull-Based Communication," in *IEEE Transactions on Communications*, vol. 70, no. 3, pp. 1606-1622, 2022.
- [15] A. Jaiswal and A. Chattopadhyay, "Minimization of Age-of-Information in Remote Sensing with Energy Harvesting," in *ISIT* 2021, pp. 3249-3254.
- [16] M. A. Abd-Elmagid and H. S. Dhillon, "Age of Information in Multi-source Updating Systems Powered by Energy Harvesting," in *IEEE Journal on Selected Areas in Information Theory*, vol. 3, no. 1, pp. 98-112, 2022.
- [17] Y. Dong, P. Fan and K. B. Letaief, "Energy Harvesting Powered Sensing in IoT: Timeliness Versus Distortion," in *IEEE Internet of Things Journal*, vol. 7, no. 11, pp. 10897-10911, 2020.
- [18] N. Nouri, D. Ardan and M.M. Feghhi, "Age of information-reliability trade-offs in energy harvesting sensor networks," arXiv preprint arXiv:2008.00987, 2020.
- [19] Z. Zhou, C. Fu, C. J. Xue et al., "Energy-Constrained Data Freshness Optimization in Self-Powered Networked Embedded Systems," in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2293-2306, 2020.
- [20] Z. Fang, J. Wang, et al., "Age of Information in Energy Harvesting Aided Massive Multiple Access Networks," in *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 5, pp. 1441-1456, 2022.
- [21] T. Zhu, J. Li, et al., "AoI Minimization Data Collection Scheduling for Battery-Free Wireless Sensor Networks," in *IEEE Transactions on Mobile Computing*, vol. 22, no. 3, pp. 1343-1355, 2023.
- [22] B. Yao, H. Gao, et al., "Maximum AoI Minimization for Target Monitoring in Battery-Free Wireless Sensor Networks," in *IEEE Transactions on Mobile Computing*, vol. 22, no. 8, pp. 4754-4772, 2023.
- [23] M. Hatami, M. Leinonen, Z. Chen, N. Pappas and M. Codreanu, "On-Demand AoI Minimization in Resource-Constrained Cache-Enabled IoT Networks With Energy Harvesting Sensors," in *IEEE Transactions on Communications*, vol. 70, no. 11, pp. 7446-7463, 2022.
- [24] T. He, K. -W. Chin, Z. Zhang, et al., "Optimizing Information Freshness in RF-Powered Multi-Hop Wireless Networks," in *IEEE Transactions on Wireless Communications*, vol. 21, no. 9, pp. 7135-7147, 2022.
- [25] Q. Chen, Z. Cai, L. Cheng, F. Wang and H. Gao, "Joint Near-Optimal Age-based Data Transmission and Energy Replenishment Scheduling at Wireless-Powered Network Edge," in *INFOCOM* 2022, pp. 770-779.
- [26] W. Jin, J. Sun, K. Chi and S. Zhang, "Deep reinforcement learning based scheduling for minimizing age of information in wireless powered sensor networks," in *Computer Communications*, vol. 191, pp. 1-10, 2022.
- [27] L. Lin, L. Ju, C. J. Xue, et al., "Work or Sleep: Freshness-Aware Energy Scheduling for Wireless Powered Communication Networks with Interference Consideration," in *DAC* 2023, pp. 1-6.
- [28] Z. Chen, N. Pappas, E. Björnson and E. G. Larsson, "Age of Information in a Multiple Access Channel with Heterogeneous Traffic and an Energy Harvesting Node," in *INFOCOM WKSHPS* 2019, pp. 662-667.
- [29] S. Leng and A. Yener, "Age of Information Minimization for an Energy Harvesting Cognitive Radio," in *IEEE Transactions on Cognitive Communications and Networking*, vol. 5, no. 2, pp. 427-439, 2019.
- [30] M. Xie, X. Jia, J. Yin, Q. Wang and M. Zhou, "Age of Information for Partial Earliest Relay Aided Short Packet Status Update With Energy Harvesting," in *IEEE Transactions on Wireless Communications*, vol. 23, no. 1, pp. 699-719, 2024.
- [31] K. Chen et al., "Distributed Energy-Adaptive Aggregation Scheduling with Coverage Guarantee For Battery-Free Wireless Sensor Networks," in *INFOCOM* 2019, pp. 1018-1026.
- [32] Q. Chen, Z. Cai, L. Cheng, et al., "Energy-collision-aware Minimum Latency Aggregation Scheduling for Energy-harvesting Sensor Networks," in *ACM Transactions on Sensor Networks*, vol. 17(4), no. 40, pp. 1-34.
- [33] J. Li et al., "Age of Aggregated Information: Timely Status Update with Over-The-Air Computation," in *GLOBECOM* 2020, pp. 1-6.
- [34] H. Pan, S. C. Liew, et al., "Coding of Multi-Source Information Streams With Age of Information Requirements," in *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1427-1440, 2021.
- [35] A. Cammarano, C. Petrioli and D. Spens, "Pro-Energy: A novel energy prediction model for solar and wind energy-harvesting wireless sensor networks," in *MASS* 2012, pp. 75-83.
- [36] C. Xu, Q. Xu, J. Wang, et al., "AoI-centric Task Scheduling for Autonomous Driving Systems," in *INFOCOM* 2022, pp. 1019-1028.
- [37] X. Wang, A. Walden, et al., "Strategies for Sensor Data Aggregation in Support of Emergency Response," in *MILCOM* 2014, pp. 1120-1126.
- [38] "Ultra Low Power Boost Converter Charger Energy Harvester," https://www.ti.com/lit/ug/sluiu654a/sluiu654a.pdf?ts=1673873261236&ref_url=https
- [39] Y. Sun, E. Uysal-Biyikoglu, R. Yates, et al., "Update or wait: How to keep your data fresh," in *INFOCOM* 2016, pp. 1-9.
- [40] L. Sigrist, A. Gomez and L. Thiele, "Dataset: Tracing indoor solar harvesting," in *DATA* 2019, pp. 47-50.
- [41] S. Yang, Y. Tahir, P. -y. Chen, A. Marshall and J. McCann, "Distributed optimization in energy harvesting sensor networks with dynamic in-network data processing," in *INFOCOM* 2016, pp. 1-9.
- [42] CC2500 Data Sheet. <https://www.ti.com/product/CC2500>
- [43] M. Cardei and DZ. Du, "Improving wireless sensor network lifetime through power aware organization," in *Wireless networks*, vol. 11, pp. 333-340, 2005.
- [44] X. Ren, W. Liang and W. Xu, "Data Collection Maximization in Renewable Sensor Networks via Time-Slot Scheduling," in *IEEE Transactions on Computers*, vol. 64, no. 7, pp. 1870-1883, 2015.
- [45] A. Yadav, M. Goonewardena, W. Ajib, O. A. Dobre and H. Elbiaze, "Energy Management for Energy Harvesting Wireless Sensors With Adaptive Retransmission," in *IEEE Transactions on Communications*, vol. 65, no. 12, pp. 5487-5498, 2017.
- [46] C. Yang, K. -W. Chin, Y. Liu, J. Zhang and T. He, "Robust Targets Coverage for Energy Harvesting Wireless Sensor Networks," in *IEEE Transactions on Vehicular Technology*, vol. 68, no. 6, pp. 5884-5892, 2019.
- [47] F. Francesco, B. Bharathan, A. Yuvraj and K. G. Rajesh, "ACES: Automatic configuration of energy harvesting sensors with reinforcement learning," in *ACM Trans. Sensor Netw.*, vol. 16, no. 4, pp. 36:1-36:31, 2020.
- [48] R. Mohammadi and Z. Shirmohammadi, "RLS2: An energy efficient reinforcement learning-based sleep scheduling for energy harvesting WBANs," in *Computer Networks*, vol. 229, no. 109781, 2023.